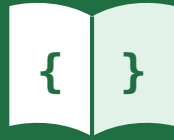


---

# Modül 09 — Hazelcast ile Dağıtık Veri

Ders Notları

Spring Boot Edu



2026-09-23

## İçindekiler

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>1. Öğrenme Hedefleri</b>                                                     | <b>2</b>  |
| <b>2. Kavramlar</b>                                                             | <b>2</b>  |
| 2.1 Hazelcast Nedir? . . . . .                                                  | 2         |
| 2.2 Embedded mi, Client-Server mı? . . . . .                                    | 2         |
| <b>3. Adım Adım Örnekler</b>                                                    | <b>3</b>  |
| 3.1 Embedded Üye . . . . .                                                      | 3         |
| 3.2 IMap . . . . .                                                              | 4         |
| 3.3 Sorgular . . . . .                                                          | 5         |
| 3.4 Hazelcast Üzerinde Spring Cache . . . . .                                   | 5         |
| 3.5 Eşzamanlı Güncellemeler: Anahtar Kilitleri ve Entry Processor'lar . . . . . | 6         |
| 3.6 Near Cache ile Client-Server . . . . .                                      | 8         |
| 3.7 İki Üyeli Cluster . . . . .                                                 | 9         |
| 3.8 Test . . . . .                                                              | 10        |
| <b>4. Sık Yapılan Hatalar ve En İyi Pratikler</b>                               | <b>10</b> |
| <b>5. Özet</b>                                                                  | <b>11</b> |
| <b>6. İleri Okuma</b>                                                           | <b>11</b> |

## 1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Embedded Hazelcast üyesi ile client-server kurulumu arasındaki farkı açıklamak
- Hazelcast'i Spring Boot'tan iki modda da başlatmak
- Dağıtık bir IMap ile veri saklamak, süresini doldurmak ve sorgulamak
- Hazelcast'i Spring'in @Cacheable anotasyonunun arkasındaki depo olarak kullanmak
- Paylaşılan veriyi eşzamanlı erişimde anahtar kilitleri ve entry processor'lar ile güvenli güncellemek
- Okumaları near cache ile hızlandırmak ve bedelini bilmek
- İki üyeli bir cluster çalıştırmak ve Hazelcast kodunu Testcontainers ile test etmek

**Ön koşullar:** Modül 08 (Spring ile önbellekleme) · **Tahmini süre:** 4 saat · **Docker** yalnızca client-server bölümü için gerekir

## 2. Kavramlar

### 2.1 Hazelcast Nedir?

Hazelcast, bellek içi bir veri ızgarasıdır (in-memory data grid): Birkaç JVM (**üye**, member) bir **cluster** oluşturur ve map, kuyruk, topic gibi veri yapılarını paylaşır. Bir map'in verisi **partition**'lara bölünür (varsayılan 271). Her partition'ın sahibi bir üyedir ve diğer üyelere yedek kopyaları vardır. Bir üye ayrıldığında yedekleri görevi devralır.

|           | Redis (modül 08)                    | Hazelcast                                                    |
|-----------|-------------------------------------|--------------------------------------------------------------|
| Çalışma   | Ayrı bir sunucu olarak              | JVM'inizin içinde (embedded) veya ayrı bir cluster olarak    |
| Veri      | String, list, set, sorted set, hash | Nesnelerden oluşan Java dostu map, kuyruk, topic...          |
| Ölçekleme | Tek düğüm veya Redis Cluster        | Üyeler katılır ve veriyi otomatik paylaşır                   |
| Hesaplama | Lua script'leri                     | Entry processor'lar ve sorgular verinin olduğu yerde çalışır |

### 2.2 Embedded mi, Client-Server mi?

| Embedded üye                                                    | Client-server                                                                            |
|-----------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Hazelcast her uygulama örneğinin içinde çalışır                 | Uygulamalar ayrı bir Hazelcast cluster'ının istemcisidir                                 |
| Ek sunucu yoktur. Veri kodun yanındadır, okumalar en hızlısıdır | Uygulamalar ve veri bağımsız ölçeklenir ve yeniden başlar                                |
| Her uygulama yeniden başlatması partition'ları taşır            | Uygulama yeniden başlatmaları veriye dokunmaz                                            |
| Tüm üyelerin aynı sınıflara ihtiyacı vardır                     | Sunucular yalnızca üzerlerinde çalışan kodun (entry processor) sınıflarına ihtiyaç duyar |

Spring Boot'ta kararı tek bir bean verir: Bir `com.hazelcast.config.Config` bean'i **embedded üye** başlatır. Bir cluster'ın bağlantı bilgileri (Docker Compose, Testcontainers veya `hazelcast-client.yaml`) ise **client** oluşturur.

#### ! Önemli

Hazelcast 5.5'ten beri **CP Subsystem** (FencedLock, IAtomicLong, ISemaphore...) bir **Enterprise** özelliğidir. Community sürümünde bu çağrılar `UnsupportedOperationException`: CP subsystem is a licensed feature fırlatır. Bu yüzden bu modül, Community sürümünde bulunan IMap anahtar kilitlerini ve entry processor'ları kullanır.

## 3. Adım Adım Örnekler

Uygulamayı başlatın. Hazelcast **embedded** çalışır, Docker gerekmez:

```
./mvnw -pl modules/09-hazelcast/lesson spring-boot:run
```

Tur (LessonTour), aşağıdaki örneklerin hepsini bir kez çalıştırır ve sonuçları yazdırır. Embedded bir üye, bir web sunucusu gibi JVM'i çalışır tutar: Ctrl+C ile durdurun.

### 3.1 Embedded Üye

Boot'un ihtiyacı olan tek şey bir Config bean'idir. HazelcastAutoConfiguration üyeyi bundan kurar ve HazelcastInstance bean'ini kaydeder:

```
@Configuration(proxyBeanMethods = false)
@Profile("!client")
public class MemberConfiguration {
```

```

@Bean
Config hazelcastConfig() {
    Config config = new Config();
    config.setClusterName("bookstore");
    config.setProperty("hazelcast.phone.home.enabled", "false");
    var join = config.getNetworkConfig().getJoin();
    join.getAutoDetectionConfig().setEnabled(false);           // a single member: do
↪ not look for others
    join.getMulticastConfig().setEnabled(false);
    config.addMapConfig(new MapConfig("prices").setTimeToLiveSeconds(600)); //
↪ cached prices expire
    return config;
}
}

```

- `setClusterName`: Yalnızca aynı adı taşıyan üyeler bir cluster oluşturur.
- Join ayarları üyelerin birbirini nasıl bulacağını belirler: multicast, TCP/IP üye listesi veya bulut keşfi (Kubernetes, AWS...). Tek bir üyenin hiçbirine ihtiyacı yoktur.
- `@Profile("!client")`: client profilinde (bölüm 3.6) uygulama üye başlatmamalıdır.

#### ★ İpucu

Config bean'i yerine classpath'e bir `hazelcast.yaml` koyabilir veya `spring.hazelcast.config` ile bir dosya gösterebilirsiniz. Boot bunu da aynı şekilde kullanır.

## 3.2 IMap

Bir `IMap`, `ConcurrentMap` gibi davranır, ancak kayıtları cluster'ın partition'larına dağılmıştır:

```

@Service
public class BookCatalog {

    private final IMap<String, Book> books;

    public BookCatalog(HazelcastInstance hazelcast) {
        this.books = hazelcast.getMap("books");           // created on first
↪ use
    }

    public void save(Book book) {
        books.set(book.isbn(), book);                   // set(): like put()
↪ without returning the old value
    }
}

```

```

public boolean addIfAbsent(Book book) {
    return books.putIfAbsent(book.isbn(), book) == null;    // atomic across
    ↪ the whole cluster
}

public void saveFor(Book book, Duration timeToLive) {
    books.set(book.isbn(), book, timeToLive.toMillis(), TimeUnit.MILLISECONDS);    //
    ↪ this entry expires on its own
}

public Optional<Book> find(String isbn) {
    return Optional.ofNullable(books.get(isbn));
}

```

- set, eski değeri döndürmeden saklar. put'a göre bir ağ gidiş-dönüşü kazandırır.
- putIfAbsent tüm cluster genelinde atomiktir.
- Dört parametrelili set, tek bir kayda kendi yaşam süresini verir. MapConfig.setTimeToLiveSeconds aynısını tüm map için yapar.
- Book sıradan bir record'dur. Hazelcast record'ları hiçbir yapılandırma olmadan **Compact serileştirme** ile serileştirir. Bu biçim kendi şemasını taşır, bu yüzden bir sunucu üyesi Book sınıfına sahip olmadan kitapları saklayabilir ve sorgulayabilir.

### 3.3 Sorgular

Sorgular her üyede, o üyenin sahip olduğu veri üzerinde paralel çalışır:

```

public Collection<Book> cheaperThan(BigDecimal limit) {
    return books.values(Predicates.lessThan("price", limit));    // runs on every member,
    ↪ in parallel
}

```

Predicates; equal, between, like, in, and, or ve daha fazlasını sunar. map.addIndex(IndexType.SORTED, "price") ile aralık sorguları artık her kaydı taramaz.

Tur çıktısı:

```

== 3.3 Query
cheaper than 90: [Effective Java, Java Puzzlers]
Spring in Action after its 2 s TTL: expired

```

### 3.4 Hazelcast Üzerinde Spring Cache

Önbellek anotasyonları modül 08'deki ile aynıdır. Yalnızca depo değişir. Bir HazelcastInstance bean'i ve classpath'te hazelcast-spring varsa Boot bir HazelcastCacheManager oluş-

turur. Her önbellek aynı adlı bir IMap'tir:

```
@Service
public class PriceService {

    private final AtomicInteger calls = new AtomicInteger();

    @Cacheable("prices")
    public BigDecimal priceOf(String isbn) {
        calls.incrementAndGet();
        sleep(300); // pretend to ask a
        ↪ slow pricing system
        return new BigDecimal("89.90");
    }
}
```

prices önbelleğinin yaşam süresi, MemberConfiguration içindeki MapConfig'ten gelir (bölüm 3.1).

```
== 3.4 Spring Cache
1st price: 89.90 in 311 ms
2nd price: 89.90 in 0 ms
```

### 3.5 Eşzamanlı Güncellemeler: Anahtar Kilitleri ve Entry Processor'lar

Stok ayırmak “miktarı oku, kontrol et, geri yaz” demektir. İki çağırana araya girerse ikisi de 1 kitap kaldığını görür ve kitap iki kez satılır. İki güvenli yol vardır.

**Anahtar kilidi.** IMap.lock(key) tüm cluster'da tek bir anahtar kilitler. Diğer anahtarlar serbest kalır:

```
public boolean reserveWithLock(String isbn, int quantity) {
    stock.lock(isbn); // cluster-wide lock on
    ↪ this key only
    try {
        int available = available(isbn);
        if (available < quantity) {
            return false;
        }
        stock.set(isbn, available - quantity);
        return true;
    } finally {
        stock.unlock(isbn); // always release, even
        ↪ after an exception
    }
}
```

```

public boolean tryReserve(String isbn, int quantity) throws InterruptedException {
    if (!stock.tryLock(isbn, 100, TimeUnit.MILLISECONDS)) {           // do not wait forever
        ↪ for a busy key
        return false;
    }
    try {
        int available = available(isbn);
        if (available < quantity) {
            return false;
        }
        stock.set(isbn, available - quantity);
        return true;
    } finally {
        stock.unlock(isbn);
    }
}

```

- Kiliti her zaman finally içinde açın. Kilit, onu alan thread'e aittir.
- Zaman aşımli tryLock, meşgul bir anahtar sonsuza kadar beklemek yerine vazgeçer.
- lock(key, leaseTime, unit), kilidi tutan hiç açmazsa diye, kira süresi sonunda kilidi kendiliğinden bırakır.

**Entry processor.** Veriyi koda getirmek yerine kodu veriye gönderin. Processor, anahtarın sahibi olan üyede çalışır ve Hazelcast anahtar başına aynı anda yalnızca bir processor çalıştırır. Kilit gerekmez ve dört yerine tek ağ gidiş-dönüşü yeterlidir:

```

public record ReserveStock(int quantity) implements EntryProcessor<String, Integer,
    ↪ Boolean> {

    @Override
    public Boolean process(Map.Entry<String, Integer> entry) {
        Integer available = entry.getValue();
        if (available == null || available < quantity) {
            return false;
        }
        entry.setValue(available - quantity);           // written back by
    ↪ Hazelcast
        return true;
    }
}

```

```

public boolean reserveWithEntryProcessor(String isbn, int quantity) {
    return Boolean.TRUE.equals(stock.executeOnKey(isbn, new ReserveStock(quantity)));
    ↪ // one round trip
}

```

StockReservationTest, 50 virtual thread'in aynı anda 10 kitap almasını dener. İki sürüm de tam olarak 10 kitap satar.

### ! Uyarı

Entry processor **üye üzerinde** çalıştırılır. Client-server modunda sınıfı sunucuların classpath'inde olmalıdır, aksi hâlde çağrı başarısız olur. Tur bu yüzden client profilinde entry processor'ı atlar. Entry processor'ları embedded kurulumlarda tercih edin veya sınıflarını cluster'a dağıtın.

## 3.6 Near Cache ile Client-Server

Uygulamayı, kök dizindeki compose.yaml dosyasındaki Hazelcast üyesinin **client**'i olarak başlatın:

```
./mvnw -pl modules/09-hazelcast/lesson spring-boot:run -Dspring-boot.run.profiles=client
```

client profili Docker Compose desteğini açar. Boot hazelcast/hazelcast'ı başlatır, HZ\_CLUSTERNAME değerini okur ve HazelcastConnectionDetails sağlar:

```
spring:
  docker:
    compose:
      enabled: true # starts hazelcast/hazelcast and provides
      ↪ HazelcastConnectionDetails
```

Yalnızca bağlantı bilgileri olsaydı client'ı Boot kendisi kurardı. Burada bir **near cache**, yani client içinde books map'inin yerel bir kopyasını eklemek için client'ı kendimiz kuruyoruz:

```
@Configuration(proxyBeanMethods = false)
@Profile("client")
public class ClientConfiguration {

    @Bean(destroyMethod = "shutdown")
    HazelcastInstance hazelcastInstance(HazelcastConnectionDetails connection) {
        ClientConfig config = connection.getClientConfig(); // address + cluster
        ↪ name from Docker Compose / Testcontainers
        config.addNearCacheConfig(new NearCacheConfig("books")
            .setInMemoryFormat(InMemoryFormat.OBJECT) // keep deserialized
            ↪ objects: fastest reads
            .setTimeToLiveSeconds(60) // bound how stale a
            ↪ local copy can get
            .setInvalidateOnChange(true)); // the cluster tells
            ↪ us when an entry changes
    }
}
```

```

        return HazelcastClient.newHazelcastClient(config);
    }
}

```

- İlk get sunucuya gider. Aynı anahtarın tekrar okunması yerel bellekten yanıtlanır. Ödevde near cache'ten bir okuma yaklaşık 1,5 µs, üyeden bir okuma yaklaşık 70 µs sürdü.
- `setInvalidateOnChange(true)`: Cluster'da bir kayıt değişince üye, client'lara kopyalarını silmelerini söyler.
- Geçersiz kılma mesajları **toplu** gönderilir: Varsayılan olarak bir üye bunları 10 saniyede bir veya 100 değişiklikte bir yollar. Bu yüzden near cache sonunda tutarlıdır (eventually consistent). Değiştiğinden çok daha sık okunan veri için kullanın.

### 3.7 İki Üyeli Cluster

TwoMemberClusterTest tek bir JVM'de iki üye başlatır. Üyeler birbirini 127.0.0.1 üzerinden TCP/IP ile bulur:

```

private static Config memberConfig() {
    Config config = new Config();
    config.setClusterName("cluster-demo"); // only members
    ↪ with this name join
    config.setProperty("hazelcast.phone.home.enabled", "false");
    config.getNetworkConfig().setPort(5801); // own port
    ↪ range, away from 5701
    var join = config.getNetworkConfig().getJoin();
    join.getAutoDetectionConfig().setEnabled(false);
    join.getMulticastConfig().setEnabled(false);
    join.getTcpIpConfig().setEnabled(true)
        .addMember("127.0.0.1:5801").addMember("127.0.0.1:5802"); // the two
        ↪ members of this demo
    config.getMapConfig("books").setBackupCount(1); // one backup
    ↪ copy on another member
    return config;
}

```

Testler üç şey gösterir:

1. İki üye de 2 üyeli bir cluster görür.
2. Bir üyeye yazılan kayıt diğerinden okunabilir. Çağırın, anahtarın hangi üyede olduğunu bilmez.
3. Bir üye kapandıktan sonra 100 kaydın hepsi yerindedir, çünkü diğer üyedeki yedekler sahip olur.

★ **İpucu**

Üretimde üyeler nadiren sabit bir üye listesi kullanır. Örneğin Kubernetes'te Hazelcast Kubernetes keşif eklentisi üyeleri Kubernetes API'si üzerinden bulur.

### 3.8 Test

- Embedded kod `@SpringBootTest` ile test edilir: Her test context'i kendi üyesini başlatır. Test üyeleri birbirini bulmasın diye join'i kapalı tutun.
- Client-server kodu Docker'daki gerçek bir üyeye karşı test edilir. Testcontainers'ın Hazelcast'e özel bir container sınıfı yoktur, bu yüzden bir `GenericContainer` kullanılır ve `@ServiceConnection` image adını ister:

```
@TestConfiguration(proxyBeanMethods = false)
static class HazelcastServer {

    static final GenericContainer<?> MEMBER = new
        ↪ GenericContainer<>("hazelcast/hazelcast:5.5.0") // as compose.yaml
        .withEnv("HZ_CLUSTERNAME", "bookstore") // Boot reads it to set
        ↪ the client's cluster name
        .withExposedPorts(5701);

    @Bean
    @ServiceConnection(name = "hazelcast/hazelcast") // generic container: tell
    ↪ Boot what it runs
    GenericContainer<?> hazelcastMember() {
        return MEMBER;
    }
}
```

! **Uyarı**

name = "hazelcast/hazelcast" olmadan test `ConnectionDetailsNotFoundException` ile başarısız olur, çünkü Boot generic bir container'ın ne çalıştırdığını bilemez.

## 4. Sık Yapılan Hatalar ve En İyi Pratikler

!! **Dikkat**

Paylaşılan veride kilit veya entry processor olmadan asla "oku, kontrol et, yaz" yapmayın. Uygulamanızın her örneği aynı kodu aynı anda çalıştırıyor olabilir.

- **Yapın:** Multicast'i kapatın ve üyelerinizi açıkça listeleyin (veya bir keşif eklentisi kullanın). Multicast, aynı ağdaki ilgisiz uygulamaların cluster'ınıza katılmasına izin verir.

- **Yapmayın:** `finally` içinde `unlock` çağırmaı unutmayın. Kilitli kalan bir anahtar sonraki tüm çağırınları bekletir.
- **Yapın:** Birden çok anahtarı sabit bir sırayla (ör. sıralı) kilitleyin. Aynı anahtarları ters sırayla kilitleyen iki çağırın sonsuza kadar birbirini bekler: deadlock (Ödev 2).
- **Yapmayın:** Tek bir kayıta çok büyük nesneler saklamayın. Üyeler katılıp ayrıldığında bir partition bütün olarak taşınır.
- **Yapın:** Önbelleklere bir yaşam süresi verin. Biraz eski veri sorun olacaksa near cache'lere kısa bir süre verin.
- **Yapmayın:** 5.5'ten beri Community sürümünde `FencedLock` veya `IAtomicLong`'un çalışmasını beklemeyin.

## 5. Özet

- Hazelcast veriyi JVM'ler arasında paylaşır. Partition'ların sahipleri ve yedekleri vardır, bu yüzden cluster bir üyenin kaybından sağ çıkar.
- Boot'ta bir Config bean'i embedded üye, `HazelcastConnectionDetails` ise client verir.
- `IMap`; kayıt başına TTL, `putIfAbsent` gibi atomik işlemler ve `Predicates` ile paralel sorgular sunar.
- `@Cacheable` değişmeden çalışır: Her önbellek bir `IMap`'tir.
- Anahtar kilitleri ve entry processor'lar eşzamanlı güncellemeleri güvenli yapar. Entry processor'lar verinin olduğu yerde çalışır.
- Near cache, tekrar eden client okumalarını yerel ve hızlı yapar, ancak yalnızca sonunda tutarlıdır.
- Testlerde embedded üyeler Docker istemez. Client-server testleri `@ServiceConnection(name = "hazelcast/hazelcast")` ile bir `GenericContainer` kullanır.

Sıradaki adım: [Ödevler](#)

## 6. İleri Okuma

- [Spring Boot — Hazelcast](#)
- [Hazelcast — Distributed Map · Locking Maps](#)
- [Hazelcast — Entry Processor](#)
- [Hazelcast — Near Cache](#)
- [Hazelcast — Compact Serialization](#)
- [Hazelcast — CP Subsystem · 5.5 Release Notes](#)